



Plataforma de Reserva de servicios



Planificación de Testing

Nivel	Tipo de Test	Herramientas	Cobertura Objetivo	Frecuencia
Unitario	Lógica de dominio, handlers CQS, validaciones	Jest / Vitest, Testing Library	$\geq 85\%$ de líneas en capa domain y application	Cada commit (CI)
Integración	Repositorios con BD real, endpoints REST	Supertest, TestContainers (PostgreSQL en Docker)	$\geq 75\%$ de flujos de API	Cada PR (CI)
Concurrencia	Prevención de doble reserva, race conditions	k6, Artillery, scripts personalizados	100 req. simultáneos $\rightarrow$ 0 dobles reservas	Previo a cada release
Seguridad	SQL Injection, XSS, autenticación JWT	OWASP ZAP, análisis estático (ESLint security)	0 vulnerabilidades críticas o altas	Cada release
E2E	Flujos completos usuario-sistema	Playwright / Cypress	$\geq 70\%$ flujos críticos (reserva, login, cancelación)	Nightly build
Carga / Estrés	Rendimiento bajo carga sostenida y pico	k6, Grafana (monitoreo durante prueba)	P95 < 2s con 500 usuarios concurrentes	Previo a producción



Fase	Actividades Principales	Entregables	Dependencias	Validación / Criterio de Éxito
Fase 0 Preparación (Sprint 0)	Configuración de repositorio GitHub. Definición de GitFlow. Configuración de Docker y Docker Compose.	Repositorio configurado. Pipelines CI/CD operativos. Entornos de desarrollo	Ninguna (punto de partida)	Pipeline CI ejecuta build exitoso en rama main. Todos los entornos responden a
	Setup de CI/CD base (GitHub Actions). Configuración de entornos (dev, staging, prod). Definición de estándares de código.	levantados. Documentación de arquitectura técnica.		health-check. Estándares de código documentados y aprobados.
Fase 1 Autenticación & Perfiles	Diseño del modelo de datos de usuarios. Implementación de endpoints de registro, login, logout y recuperación de contraseña. Generación y validación de JWT (RS256). Hash bcrypt. Middleware de autenticación y autorización por roles. Módulo de perfil de usuario.	API de autenticación funcional. Middleware JWT integrado. Tests unitarios y de integración del módulo. Documentación de API (OpenAPI).	Fase 0 completada	Login exitoso retorna JWT válido. Middleware rechaza requests sin token válido (HTTP 401/403). Contraseñas almacenadas como hash bcrypt. Cobertura de tests >= 80%.



Fase 2 Disponibilidad & Agenda	Modelo de datos de agenda, slots y servicios. Implementación CQS: endpoints de consulta separados de comandos. Lógica de cálculo dinámico de slots disponibles. Configuración de horarios y bloqueos manuales. Excepciones de agenda. Caché de disponibilidad.	API de disponibilidad funcional. Módulo de agenda del administrador. Caché de slots implementado. Tests de carga básicos.	Fase 1 completada (JWT requerido para endpoints de escritura)	Consulta de disponibilidad responde en < 2s (RNF-03). CQS correctamente implementado (endpoints Query no modifican estado). Tests de unidad sobre lógica de cálculo de slots al 85%.
Fase 3 Agendamiento & Concurrencia	Modelo de datos de citas y estados. Implementación de reserva, cancelación y reprogramación. Mecanismo anti-doble reserva (bloqueo pesimista SELECT FOR UPDATE). Gestión de estados de cita. Integración con módulo de disponibilidad. Pruebas de concurrencia.	API de agendamiento funcional. Tests de concurrencia (simulación de 50+ usuarios simultáneos). Documentación del mecanismo de bloqueo.	Fase 2 completada (disponibilidad como dependencia directa)	Cero dobles reservas en tests de concurrencia con 100 requests simultáneos sobre el mismo slot (RF-22, RNF-02). Transacciones ACID verificadas. Estados de cita correctamente gestionados.

Fase 4 Notificaciones	Diseño de cola de mensajes asíncrona. Implementación de servicio de correo electrónico (SMTP). Templates de notificación (confirmación, recordatorio, cancelación, reprogramación). Integración mediante eventos de dominio. Sistema de reintentos con backoff exponencial.	Servicio de notificaciones desacoplado. Templates de email implementados. Log de notificaciones. Sistema de reintentos.	Fase 3 completada (eventos de dominio de citas como fuente)	Notificaciones de confirmación enviadas en < 30s tras la reserva. Tasa de entrega >= 98% en ambiente de staging. Fallos encolados y reintentados automáticamente.
Fase 5 Frontend & Integración	Implementación de SPA React/TypeScript. Componentes de calendario, formularios de reserva, panel de administración. Integración con todos los endpoints REST. Gestión de estado y autenticación JWT en cliente. Pruebas E2E. Optimización de performance.	SPA completamente funcional. Flujo de reserva en <= 3 pasos (RNF-20). Pruebas E2E automatizadas. Build de producción optimizado.	Fases 1-4 completadas (APIs como contrato)	Flujo de reserva completado en <= 3 pasos (RNF-20). Diseño responsive verificado en mobile/tablet/desktop (RNF-19). Pruebas E2E al 70% de cobertura de flujos críticos.



Fase 0: **Construcción del Proyecto**

Codigo a Probar



 docker-compose.yml

```

1  version: '3.9'
2  | Ctrl+L for Agent
   | Run All Services
3  services:
   | Run Service
4  postgres:
5    image: postgres:15-alpine
6    container_name: uni_booking_db
7    environment:
8      POSTGRES_USER: ${POSTGRES_USER}
9      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
10     POSTGRES_DB: ${POSTGRES_DB}
11    ports:
12     - "5432:5432"
13    volumes:
14     - postgres-data:/var/lib/postgresql/data
15    healthcheck:
16     test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
17     interval: 10s
18     timeout: 5s
19     retries: 5
20    restart: unless-stopped
21
   | Run Service
22  redis:
23    image: redis:7-alpine
24    container_name: uni_booking_redis
25    command: redis-server --requirepass ${REDIS_PASSWORD} --appendonly yes
26    ports:
27     - "6379:6379"
28    volumes:
29     - redis-data:/data
30    healthcheck:
31     test: ["CMD", "redis-cli", "--pass", "${REDIS_PASSWORD}", "ping"]
32     interval: 10s
33    restart: unless-stopped
34

```

src > backend >  Dockerfile > ...

```

1  # Etapa 1: Builder (instala devDependencies y compila TypeScript)
2  FROM node:20-alpine AS builder
3  WORKDIR /app
4  COPY package*.json ./
5  RUN npm ci && npm cache clean --force
6  COPY . .
7  RUN npm run build
8
9  # Etapa 2: Producción (solo dependencias de producción + artefactos compilados)
10 FROM node:20-alpine AS production
11 RUN addgroup -g 1001 -S nodejs && adduser -S nodejs -u 1001
12 WORKDIR /app
13
14 # Copiar package.json para instalar solo dependencias de producción
15 COPY package*.json ./
16 RUN npm ci --omit=dev && npm cache clean --force
17
18 # Copiar los artefactos compilados desde el builder
19 COPY --from=builder --chown=nodejs:nodejs /app/dist ./dist
20
21 USER nodejs
22 EXPOSE 3000
23
24 HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
25 | CMD wget -qO- http://localhost:3000/api/health || exit 1
26
27 CMD ["node", "dist/main.js"]

```


Resultado

```
PS C:\Users\jose\Desktop\Proyectos\Cursos\Proy_SW505_2026_1> docker compose up
[+] Running 5/5
 ✓ Network proy_sw505_2026_1_default    Created           0.2s
 ✓ Container uni_booking_redis          Created           0.5s
 ✓ Container uni_booking_db             Created           0.5s
 ✓ Container uni_booking_api            Created           0.3s
 ✓ Container uni_booking_spa            Created           0.3s
```

```
localhost:3000/api/health
WhatsApp Traductor de Google ORCE | Inicio
Impresión con sangría ☐
{"status":"UP","timestamp":"2026-06-04T21:00:01.445Z"}
```

localhost

WhatsApp Traductor de Google ORCE | Inicio LEAD UNI | Comun... Lectura LEAD UNI - Bc

Sistema de Gestión de Reservas - Arquitectura CQS

El frontend SPA se ha compilado y desplegado correctamente.



Fase 1: **Autenticación y Perfiles**



```
import { InvalidUserIdException } from '../exceptions/invalid-user-id.exception';

const UUID_REGEX = /^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i;

export class TenantId {
  private constructor(private readonly _value: string) {}

  static create(value: string): TenantId {
    const normalizedValue = value.trim();

    if (!UUID_REGEX.test(normalizedValue)) {
      throw new InvalidUserIdException(value);
    }

    return new TenantId(normalizedValue);
  }

  get value(): string {
    return this._value;
  }

  equals(other: TenantId): boolean {
    return this._value === other.value;
  }
}
```

```
import { InvalidUserIdException } from '../exceptions/invalid-user-id.exception';

const UUID_REGEX = /^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i;

export class UserId {
  private constructor(private readonly _value: string) {}

  static create(value: string): UserId {
    const normalizedValue = value.trim();

    if (!UUID_REGEX.test(normalizedValue)) {
      throw new InvalidUserIdException(value);
    }

    return new UserId(normalizedValue);
  }
}
```



Codigo para Probar

```
export class User {
  private constructor(
    private _id: UserId,
    private _email: Email,
    private _passwordHash: PasswordHash,
    private _fullName: FullName,
    private _role: UserRole,
    private _status: UserStatus,
    private _createdAt: Date,
    private _updatedAt: Date,
  ) {}

  static create(props: CreateUserProps): User {
    return new User(
      User.toUserId(props.id),
      User.toEmail(props.email),
      User.toPasswordHash(props.passwordHash),
      User.toFullName(props.fullName),
      User.toUserRole(props.role ?? 'CLIENT'),
      User.toUserStatus(props.status ?? 'PENDING'),
      props.createdAt ?? new Date(),
      props.updatedAt ?? new Date(),
    );
  }
}
```

```
describe('User Entity', () => {
  it('should create and reconstitute a user entity', () => {
    const user = User.create({
      id: validUuid1,
      email: 'user@example.com',
      passwordHash: 'b'.repeat(60),
      fullName: 'Test User',
    });

    expect(user.id.value).toBe(validUuid1);
    expect(user.email.value).toBe('user@example.com');
    expect(user.fullName.value).toBe('Test User');
    expect(user.role.value).toBe('CLIENT');
    expect(user.status.value).toBe('PENDING');
  });
});
```

Codigo a Probar



```
import { InvalidUserIdException } from '../exceptions/invalid-user-id.exception';
```

```
const UUID_REGEX = /^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i;
```

```
export class TenantId {  
  private constructor(private readonly _value: string) {}
```

```
  static create(value: string): TenantId {  
    const normalizedValue = value.trim();  
  
    if (!UUID_REGEX.test(normalizedValue)) {  
      throw new InvalidUserIdException(value);  
    }  
  
    return new TenantId(normalizedValue);  
  }
```

```
  get value(): string {  
    return this._value;  
  }
```

```
  equals(other: TenantId): boolean {  
    return this._value === other.value;  
  }
```

```
}
```

```
import { InvalidUserIdException } from '../exceptions/invalid-user-id.exception';
```

```
const UUID_REGEX = /^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i;
```

```
export class UserId {  
  private constructor(private readonly _value: string) {}
```

```
  static create(value: string): UserId {  
    const normalizedValue = value.trim();
```

```
    if (!UUID_REGEX.test(normalizedValue)) {  
      throw new InvalidUserIdException(value);  
    }
```

```
    return new UserId(normalizedValue);  
  }
```


Codigo para Probar

[Home](#)[About Us](#)[Contact](#)

```
import { CustomerRepository } from '../../domain/repositories/customer.repository';
import { CustomerNotFoundException } from '../../domain/exceptions/customer-not-found.exception';
import { UserId } from '../../domain/value-objects/user-id.vo';
import { Email } from '../../domain/value-objects/email.vo';
import { UpdateCustomerCommand } from './update-customer.command';
import { CustomerResponseDto } from './create-customer.command-handler';
import { Customer } from '../../domain/entities/customer.entity';

export class UpdateCustomerCommandHandler {
  constructor(private readonly customerRepository: CustomerRepository) {}

  async execute(command: UpdateCustomerCommand): Promise<CustomerResponseDto> {
    const customer = await this.customerRepository.findByUserId(UserId.create(command.userId));

    if (!customer) {
      throw new CustomerNotFoundException(command.userId);
    }

    customer.updateProfile({
      firstName: command.firstName,
      lastName: command.lastName,
      email: command.email ? Email.create(command.email) : undefined,
      phone: command.phone,
      timezone: command.timezone,
      preferences: command.preferences,
      consentSigned: command.consentSigned,
    });

    const savedCustomer = await this.customerRepository.update(customer);
    return this.toDto(savedCustomer);
  }
}
```




Codigo para Probar

```
describe('Phase 1 Extra Command Handlers - Equivalencia, Límites y Caminos', () => {  
  const validUuid = '123e4567-e89b-12d3-a456-426614174000';  
  
  describe('UpdateCustomerCommandHandler', () => {  
    let mockCustomerRepo: any;  
    let handler: UpdateCustomerCommandHandler;  
  
    beforeEach(() => {  
      mockCustomerRepo = {  
        findByIdById: jest.fn(),  
        update: jest.fn(),  
      };  
      handler = new UpdateCustomerCommandHandler(mockCustomerRepo);  
    });  
  });  
});
```



Codigo para Probar

```
it('[Feliz] Debe actualizar un cliente cuando se proveen todos los campos opcionales', async () => {
  const mockCustomer = Customer.create({
    id: validUuid,
    tenantId: validUuid,
    userId: validUuid,
    firstName: 'Old',
    lastName: 'Name',
    email: 'old@test.com',
    phone: '123456',
    timezone: 'UTC',
    consentSigned: true,
  });
  mockCustomerRepo.findByUserId.mockResolvedValue(mockCustomer);
  mockCustomerRepo.update.mockResolvedValue(mockCustomer);

  const result = await handler.execute({
    userId: validUuid,
    firstName: 'New',
    lastName: 'Name',
    email: 'new@test.com',
    phone: '123456',
    timezone: 'UTC',
  });

  expect(result.firstName).toBe('New');
  expect(result.email).toBe('new@test.com');
  expect(mockCustomerRepo.update).toHaveBeenCalled();
});
```




Resultados

```
> uni-booking-api@1.0.0 test:unit
> jest --config jest.config.cjs
```

```
PASS tests/unit/application/phase2-availability.spec.ts
PASS tests/unit/application/phase1-commands-extra.spec.ts
PASS tests/unit/domain/value-objects/phase2-value-objects.spec.ts
PASS tests/unit/domain/entities/phase1-entities.spec.ts
PASS tests/unit/domain/entities/phase2-domain.spec.ts
PASS tests/unit/application/phase2-handlers.spec.ts
PASS tests/unit/application/phase1-handlers.spec.ts
PASS tests/unit/domain/value-objects/phase1-value-objects.spec.ts
```

A worker process has failed to exit gracefully and has been force exited. This is likely caused by tests leaking due to improper teardown. Try running with `--detectOpenHandles` to find leaks. Active timers can also cause this, ensure that `.unref()` was called on them.

Test Suites: 8 passed, 8 total

Tests: 92 passed, 92 total

Snapshots: 0 total

Time: 5.78 s

Ran all test suites.

PS C:\Users\iose\Desktop\Proyectos\Cursos\Prov SW505 2026 1\src\backend>



Codigo a Probar

```
import { DataSource, Not } from 'typeorm';
import { UserRepository } from '../../../domain/repositories/user.repository';
import { User } from '../../../domain/entities/user.entity';
import { Email } from '../../../domain/value-objects/email.vo';
import { UserId } from '../../../domain/value-objects/user-id.vo';
import { UserStatus } from '../../../domain/value-objects/user-status.vo';
import { TypeOrmUserEntity } from './entities/typeorm-user.entity';

export class TypeOrmUserRepository implements UserRepository {
  constructor(private readonly dataSource: DataSource) {}

  private get repository() {
    return this.dataSource.getRepository(TypeOrmUserEntity);
  }

  async findById(id: UserId): Promise<User | null> {
    const entity = await this.repository.findOne({ where: { id: id.value } });
    return entity ? this.toDomain(entity) : null;
  }

  async findByEmail(email: Email): Promise<User | null> {
    const entity = await this.repository.findOne({ where: { email: email.value } });
    return entity ? this.toDomain(entity) : null;
  }
}
```

Codigo para Probar



```
import { Test, TestingModule } from '@nestjs/testing';
import { INestApplication } from '@nestjs/common';
import request from 'supertest';
import { DataSource } from 'typeorm';
import { AppModule } from '../../src/app.module';
import { INFRASTRUCTURE_TOKENS } from '../../src/infrastructure/shared/infrastructure.tokens';
import { startTestDatabase, stopTestDatabase, clearDatabase } from '../helpers/db-test-helper';
import { TypeOrmUserRepository } from '../../src/infrastructure/persistence/typeorm/typeorm-user.repository';
import { User } from '../../src/domain/entities/user.entity';
```

```
describe('AuthController Integration Tests', () => {
  let app: INestApplication;
  let dataSource: DataSource;
  let userRepository: TypeOrmUserRepository;

  beforeAll(async () => {
    dataSource = await startTestDatabase();
    userRepository = new TypeOrmUserRepository(dataSource);

    const moduleFixture: TestingModule = await Test.createTestingModule({
      imports: [AppModule],
    })
      .overrideProvider(INFRASTRUCTURE_TOKENS.DATA_SOURCE)
      .useValue(dataSource)
      .compile();

    app = moduleFixture.createNestApplication();
    await app.init();
  });
```



Codigo para Probar

```
it('POST /auth/register should register a user successfully', async () => {
  const response = await request(app.getHttpServer())
    .post('/auth/register')
    .send({
      email: 'test@example.com',
      password: 'securePassword123',
      fullName: 'Test User',
    })
    .expect(201);

  expect(response.body.id).toBeDefined();

  const foundUser = await userRepository.findByEmail(
    await userRepository.findById(response.body.id).then((u) => u!.email)
  );
  expect(foundUser).not.toBeNull();
  expect(foundUser!.email.value).toBe('test@example.com');
  expect(foundUser!.role.isClient()).toBe(true);
});
```



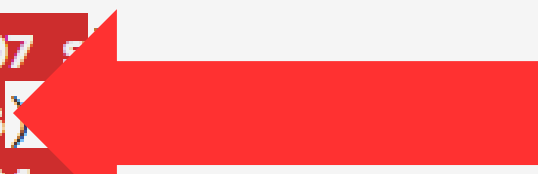
Resultados

```
PS C:\Users\jose\Desktop\Proyectos\Cursos\Proy_SW505_2026_1\src\backend> npm run test:integration
```

```
> uni-booking-api@1.0.0 test:integration
```

```
> jest --config jest.integration.config.cjs --runInBand
```

```
PASS tests/integration/http/booking.controller.spec.ts (21.207 s)
PASS tests/integration/http/auth.controller.spec.ts (13.993 s)
PASS tests/integration/http/profile.controller.spec.ts (13.584 s)
PASS tests/integration/http/companies.controller.spec.ts (14.386 s)
PASS tests/integration/http/customers.controller.spec.ts (11.915 s)
PASS tests/integration/persistence/typeorm-user.repository.spec.ts (9.703 s)
PASS tests/integration/persistence/typeorm-customer.repository.spec.ts (8.855 s)
```



```
Test Suites: 7 passed, 7 total
```

```
Tests: 18 passed, 18 total
```

```
Snapshots: 0 total
```

```
Time: 94.381 s, estimated 95 s
```

```
Ran all test suites.
```




Fase 2:

Disponibilidad y Agenda

Codigo a Probar

```
import { randomUUID } from 'crypto';
import { ResourceId } from '../value-objects/resource-id.vo';
import { ScheduleSlot } from '../entities/schedule-slot.entity';
import { IResourceRepository } from '../repositories/resource.repository';
import { TimeRange } from '../value-objects/time-range.vo';

export class SlotGeneratorService {
  constructor(private readonly resourceRepository: IResourceRepository) {}

  async generateSlotsForDateRange(
    resourceId: ResourceId,
    startDate: Date,
    endDate: Date,
    serviceDurationMinutes: number
  ): Promise<ScheduleSlot[]> {
    const resource = await this.resourceRepository.findById(resourceId);
    if (!resource) {
      throw new Error(`Resource not found: ${resourceId.value}`);
    }

    const rules = await this.resourceRepository.findAvailabilityRulesByResourceId(resourceId);
    const timeOffs = await this.resourceRepository.findTimeOffByResourceId(resourceId);
    const branchExceptions = await this.resourceRepository.findBranchExceptionsByBranchId(resource.branchId);

    const rulesWithBreaks = await Promise.all(
      rules.map(async (rule) => {
        const breaks = await this.resourceRepository.findBreaksByAvailabilityRuleId(rule.id);
        return { rule, breaks };
      })
    );

    const generatedSlots: ScheduleSlot[] = [];

    const formatDateKey = (date: Date): string => {
      return date.toISOString().split('T')[0];
    };
  }
}
```





```
const current = new Date(startDate);
const end = new Date(endDate);

const startMidnight = new Date(
  Date.UTC(current.getUTCFullYear(), current.getUTCMonth(), current.getUTCDate(), 0, 0, 0, 0)
);
const endMidnight = new Date(
  Date.UTC(end.getUTCFullYear(), end.getUTCMonth(), end.getUTCDate(), 23, 59, 59, 999)
);

const loopDate = new Date(startMidnight);

const toMinutes = (timeStr: string): number => {
  const [h, m] = timeStr.split(':').map(Number);
  return h * 60 + m;
};

while (loopDate <= endMidnight) {
  const jsDay = loopDate.getUTCDay();
  const dayOfWeekMapped = jsDay === 0 ? 7 : jsDay;

  const activeRuleWrapper = rulesWithBreaks.find(({ rule }) => {
    return rule.dayOfWeek === dayOfWeekMapped && rule.isValidOn(loopDate);
  });
```




```
if (activeRuleWrapper) {
  const { rule, breaks } = activeRuleWrapper;

  const ruleStartMins = toMinutes(rule.shift.startTime);
  const ruleEndMins = toMinutes(rule.shift.endTime);

  let currentSlotStartMins = ruleStartMins;

  while (true) {
    const currentSlotEndMins = currentSlotStartMins + serviceDurationMinutes;
    if (currentSlotEndMins > ruleEndMins) {
      break;
    }

    const slotStart = new Date(
      Date.UTC(
        loopDate.getUTCFullYear(),
        loopDate.getUTCMonth(),
        loopDate.getUTCDate(),
        Math.floor(currentSlotStartMins / 60),
        currentSlotStartMins % 60,
        0,
        0
      )
    );
  }
}
```



```
const slotEnd = new Date(  
  Date.UTC(  
    loopDate.getUTCFullYear(),  
    loopDate.getUTCMonth(),  
    loopDate.getUTCDate(),  
    Math.floor(currentSlotEndMins / 60),  
    currentSlotEndMins % 60,  
    0,  
    0  
  )  
);  
  
const slotRange = TimeRange.create(slotStart, slotEnd);  
  
const overlapsBreak = breaks.some((brk) => {  
  const bStartMins = toMinutes(brk.period.startTime);  
  const bEndMins = toMinutes(brk.period.endTime);  
  return currentSlotStartMins < bEndMins && bStartMins < currentSlotEndMins;  
});  
  
const overlapsTimeOff = timeOffs.some((to) => {  
  const toRange = TimeRange.create(new Date(to.startDatetime), new Date(to.endDatetime));  
  return slotRange.overlaps(toRange);  
});
```



```
const overlapsBranchException = branchExceptions.some((exc) => {
  if (exc.isFullDay) {
    const excDateKey = formatDateKey(new Date(exc.startDatetime));
    const currentDayKey = formatDateKey(loopDate);
    return excDateKey === currentDayKey;
  }
  const excRange = TimeRange.create(new Date(exc.startDatetime), new Date(exc.endDatetime));
  return slotRange.overlaps(excRange);
});
```

```
if (!overlapsBreak && !overlapsTimeOff && !overlapsBranchException) {
  const slot = ScheduleSlot.create({
    id: randomUUID(),
    resourceId,
    startsAt: slotStart,
    endsAt: slotEnd,
    status: 'AVAILABLE',
  });
  generatedSlots.push(slot);
}
```

```
currentSlotStartMins = currentSlotEndMins;
```

```
loopDate.setUTCDate(loopDate.getUTCDate() + 1);
```

```
return generatedSlots;
```

```
}
```



```
import { SlotGeneratorService } from '../../../src/domain/services/slot-generator.service';
import { GetAvailableSlotsQueryHandler } from '../../../src/application/queries/get-available-slots.query-handler';
import { TypeOrmAgendaSnapshotRepository } from '../../../src/infrastructure/persistence/typeorm/typeorm-agenda-snapshot.repository';

import { Resource } from '../../../src/domain/entities/resource.entity';
import { ResourceAvailabilityRule } from '../../../src/domain/entities/resource-availability-rule.entity';
import { ResourceBreak } from '../../../src/domain/entities/resource-break.entity';
import { ResourceId } from '../../../src/domain/value-objects/resource-id.vo';
import { BranchId } from '../../../src/domain/value-objects/branch-id.vo';
import { ResourceAvailabilityRuleId } from '../../../src/domain/value-objects/availability-rule-id.vo';
import { ResourceBreakId } from '../../../src/domain/value-objects/break-id.vo';

describe('Phase 2 Availability Engine & Query Handler Unit Tests', () => {
  const validUuid1 = '123e4567-e89b-12d3-a456-426614174001';
  const validUuid2 = '123e4567-e89b-12d3-a456-426614174002';
  const validUuid3 = '123e4567-e89b-12d3-a456-426614174003';

  let mockResourceRepo: any;
  let mockSnapshotRepo: any;
  let mockDataSource: any;

  Ctrl+L for Agent
  beforeEach(() => {
    mockResourceRepo = {
      findById: jest.fn(),
      findByBranchId: jest.fn(),
      findAvailabilityRulesByResourceId: jest.fn(),
      findBreaksByAvailabilityRuleId: jest.fn(),
      findTimeOffByResourceId: jest.fn(),
      findBranchExceptionsByBranchId: jest.fn(),
    };
    mockSnapshotRepo = {
      findByResourceIdAndDate: jest.fn(),
      upsert: jest.fn(),
    };
    mockDataSource = {
      query: jest.fn(),
    };
  });
});
```



```
describe('SlotGeneratorService', () => {
  it('should generate slots for date range excluding breaks, time off, and branch exceptions', async () => {
    const service = new SlotGeneratorService(mockResourceRepo);

    const resource = Resource.create({
      id: validUuid1,
      tenantId: validUuid2,
      branchId: validUuid3,
      typeId: validUuid1,
      name: 'Dr. John Doe',
      capacity: 1,
    });

    const rule = ResourceAvailabilityRule.create({
      id: validUuid1,
      resourceId: validUuid1,
      dayOfWeek: 1, // Monday
      startTime: '09:00',
      endTime: '11:00',
      validFrom: new Date('2026-06-01T00:00:00Z'),
      validTo: new Date('2026-06-30T23:59:59Z'),
    });

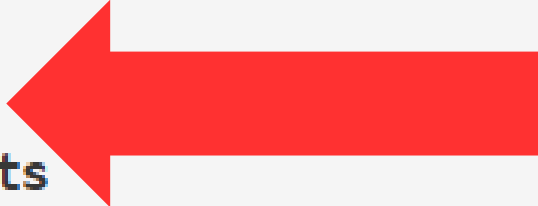
    const brk = ResourceBreak.create({
      id: validUuid1,
      availabilityRuleId: validUuid1,
      startTime: '09:30',
      endTime: '10:00',
    });
  });
});
```




Resultados

```
> uni-booking-api@1.0.0 test:unit
> jest --config jest.config.cjs
```

```
PASS tests/unit/application/phase2-availability.spec.ts
PASS tests/unit/application/phase1-commands-extra.spec.ts
PASS tests/unit/domain/value-objects/phase2-value-objects.spec.ts
PASS tests/unit/domain/entities/phase1-entities.spec.ts
PASS tests/unit/domain/entities/phase2-domain.spec.ts
PASS tests/unit/application/phase2-handlers.spec.ts
PASS tests/unit/application/phase1-handlers.spec.ts
PASS tests/unit/domain/value-objects/phase1-value-objects.spec.ts
```



A worker process has failed to exit gracefully and has been force exited. This is likely caused by tests leaking due to improper teardown. Try running with `--detectOpenHandles` to find leaks. Active timers can also cause this, ensure that `.unref()` was called on them.

Test Suites: 8 passed, 8 total

Tests: 92 passed, 92 total

Snapshots: 0 total

Time: 5.78 s

Ran all test suites.

PS C:\Users\iose\Desktop\Proyectos\Cursos\Prov SW505 2026 1\src\backend>



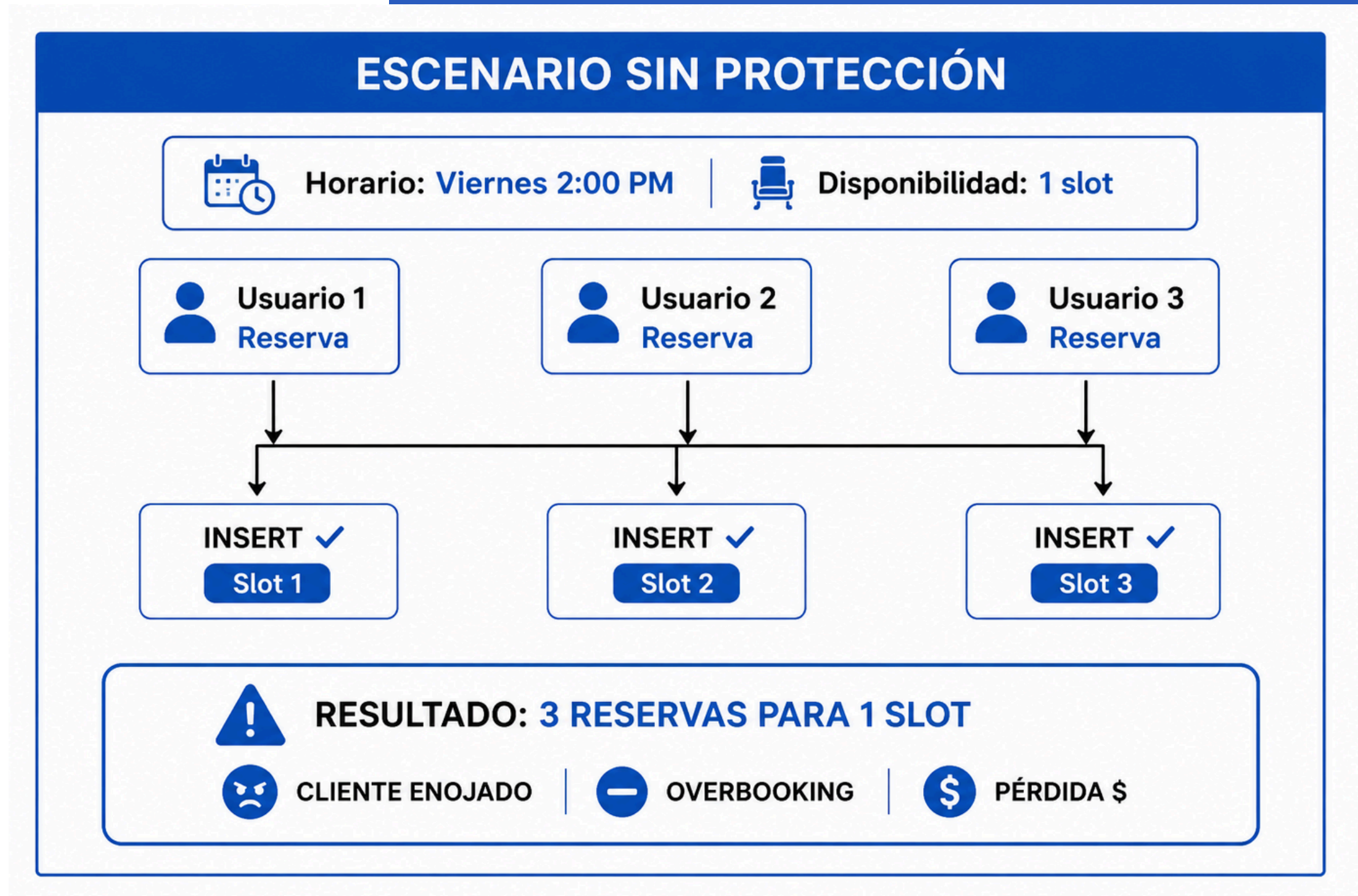
Fase 3:

Agendamiento y concurrencia

1. Problema de Negocio y Riesgo de Concurrencia

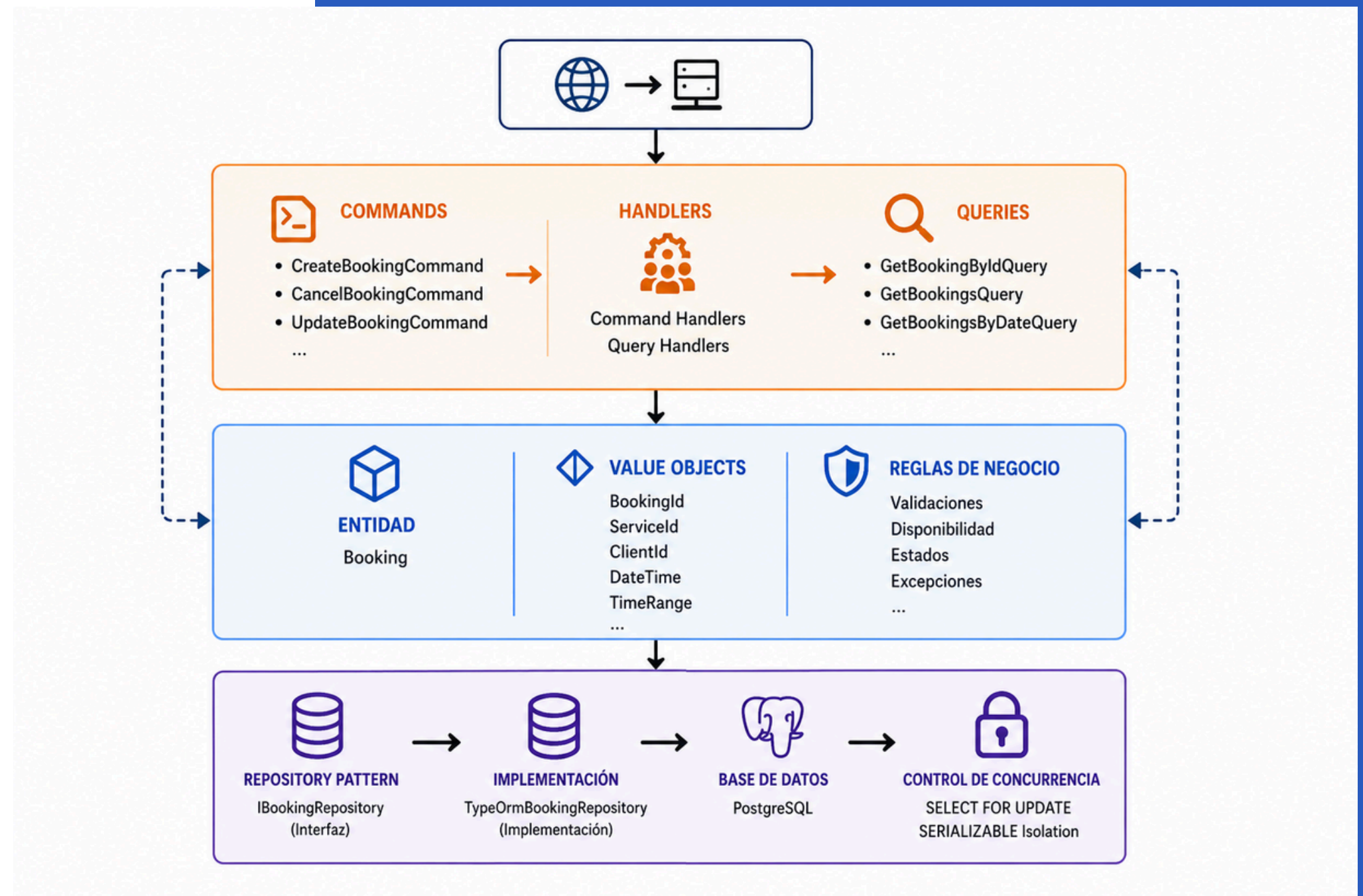
Riesgos

- Doble reserva del mismo horario
Crítico / Alta
- Conflicto con proveedor
Alto / Alta
- Pérdida financiera
Medio-Alto
- Cliente llega a cita inexistente
Medio



2. Arquitectura DDD + CQRS Aplicada al Módulo Booking

- ◆ Domain: reglas de negocio y validaciones.
- ◆ Application: Commands, Queries y Handlers.
- ◆ Infrastructure: PostgreSQL, TypeORM y Repository Pattern.
- ◆ Concurrencia: SELECT FOR UPDATE + SERIALIZABLE para prevenir doubles reservas.

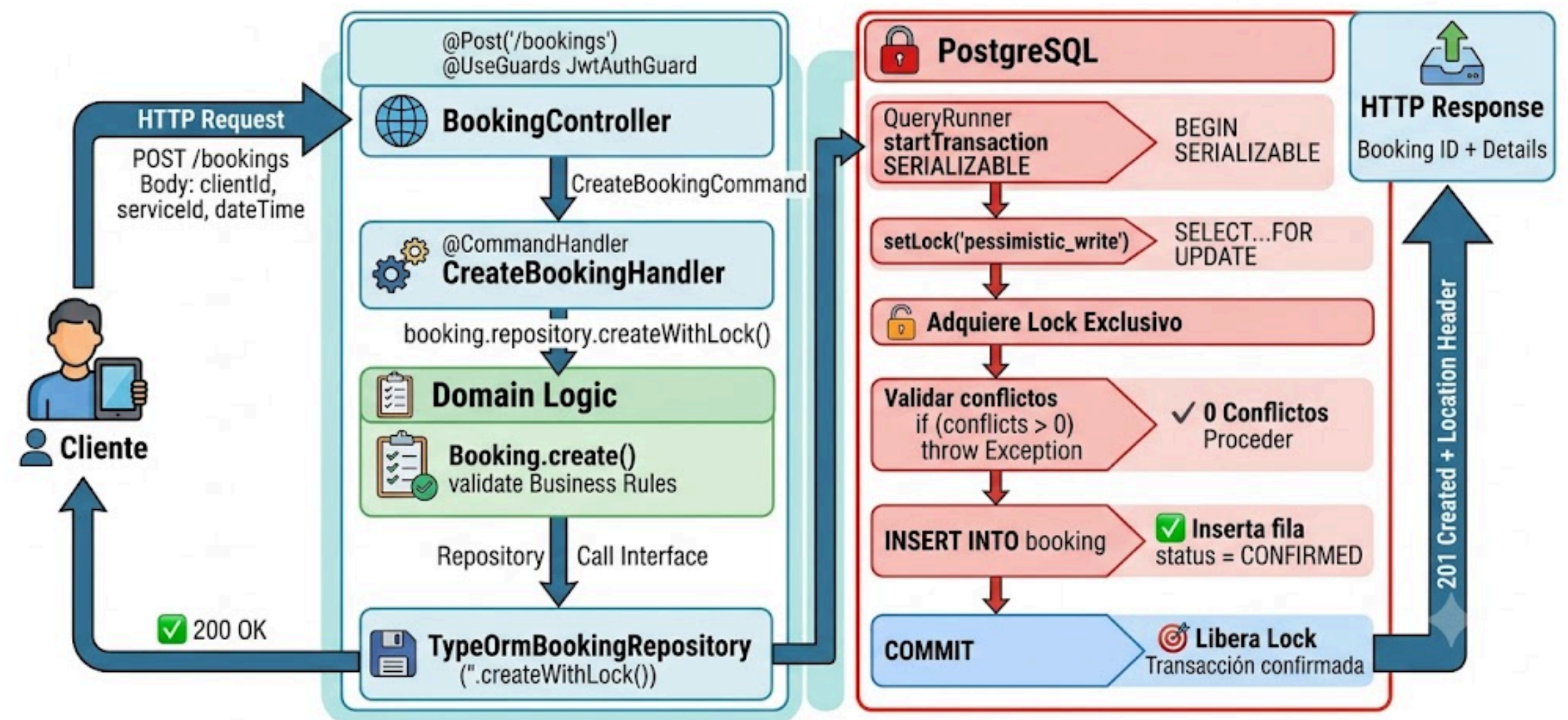


3. Flujo Completo de una Reserva

La solicitud de reserva sigue un flujo estructurado desde el controlador hasta PostgreSQL, aplicando CQRS y reglas de dominio. La base de datos asegura consistencia mediante transacciones y control de concurrencia, evitando reservas duplicadas o conflictos de horario.

DIAPPOSITIVA 4: Flujo Completo de una Reserva - Request a PostgreSQL

End-to-End: Anatomía de una Solicitud de Reserva desde REST hasta la Base de Datos



Se valida la creación y reconstitución de la entidad User, asegurando la correcta inicialización de sus atributos y el uso de value objects para garantizar la integridad del dominio.

```
describe('User Entity', () => {  
  it('should create and reconstitute a user entity', () => {  
    const user = User.create({  
      id: validUuid1,  
      email: 'user@example.com',  
      passwordHash: 'b'.repeat(60),  
      fullName: 'Test User',  
    });  
  });  
});
```

Se prueban las transiciones de estado de la entidad User, verificando que las reglas de negocio se respeten correctamente al cambiar entre estados como PENDING, ACTIVE y BLOCKED.

```
it('should manage status transitions correctly', () => {  
  const user = User.create({  
    id: validUuid1,  
    email: 'user@example.com',  
    passwordHash: 'b'.repeat(60),  
    fullName: 'Test User',  
    status: 'PENDING',  
  });  
  
  // PENDING -> ACTIVE (Allowed)  
  user.activate();  
  expect(user.status.isActive()).toBe(true);  
  
  // ACTIVE -> BLOCKED (Allowed)  
  user.block();  
  expect(user.status.isBlocked()).toBe(true);  
  
  // BLOCKED -> ACTIVE (Allowed)  
  user.activate();  
  expect(user.status.isActive()).toBe(true);  
});
```

Se valida la generación de slots de disponibilidad de recursos, asegurando que el sistema excluye correctamente bloques de descanso, tiempos no disponibles y excepciones de la sucursal.

```
describe('SlotGeneratorService', () => {
  it('should generate slots for date range excluding breaks, time off, and branch exceptions',
    const service = new SlotGeneratorService(mockResourceRepo);

    const resource = Resource.create({
      id: validUuid1,
      tenantId: validUuid2,
      branchId: validUuid3,
      typeId: validUuid1,
      name: 'Dr. John Doe',
      capacity: 1,
    });
  });
```

Validamos el registro de usuarios, asegurando la creación correcta del cliente y el uso del hash de contraseña

```
describe('RegisterUserCommandHandler', () => {
  it('should register a new client user successfully', async () => {
    const handler = new RegisterUserCommandHandler(mockUserRepo, mockHasher);
    mockUserRepo.findByEmail.mockResolvedValue(null);

    const result = await handler.execute({
      email: 'client@example.com',
      password: 'securePassword123',
      fullName: 'Client User',
    });

    expect(result.id).toBeDefined();
    expect(mockUserRepo.save).toHaveBeenCalledTimes(1);
    expect(mockHasher.hash).toHaveBeenCalledWith('securePassword123');
  });
});
```

Validamos la creación de servicios dentro del sistema, asegurando que se registren correctamente con duración y precio base.

```
describe('ConfigureResourceAvailabilityCommandHandler', () => {
  it('should configure availability rules and breaks successfully', async () => {
    const handler = new ConfigureResourceAvailabilityCommandHandler(mockResourceRepo);

    const resource = Resource.create({
      id: validUuid1,
      tenantId: validUuid2,
      branchId: validUuid3,
      typeId: validUuid1,
      name: 'Dr. John Doe',
      capacity: 1,
    });
    mockResourceRepo.findById.mockResolvedValue(resource);

    const result = await handler.execute({
      resourceId: validUuid1,
      rules: [
        {
          dayOfWeek: 1, // Monday
          startTime: '08:00',
          endTime: '17:00',
          breaks: [
            {
              startTime: '13:00',
              endTime: '14:00',
            },
          ],
        },
      ],
    });

    expect(result.id).toBe(validUuid1);
    expect(mockResourceRepo.saveAvailabilityRule).toHaveBeenCalledTimes(1);
    expect(mockResourceRepo.saveBreak).toHaveBeenCalledTimes(1);
  });
});
```

```
describe('CreateServiceCommandHandler', () => {
  it('should create and save a service successfully', async () => {
    const handler = new CreateServiceCommandHandler(mockServiceRepo);

    const result = await handler.execute({
      tenantId: validUuid1,
      categoryId: validUuid2,
      name: 'Limpieza Dental',
      baseDurationMinutes: 45,
      basePrice: 150.0,
    });

    expect(result.id).toBeDefined();
    expect(mockServiceRepo.save).toHaveBeenCalledTimes(1);

    const savedService = mockServiceRepo.save.mock.calls[0][0];
    expect(savedService.name).toBe('Limpieza Dental');
    expect(savedService.baseDurationMinutes).toBe(45);
    expect(savedService.basePrice).toBe(150.0);
  });
});
```

Se configura la disponibilidad de un recurso definiendo horarios de trabajo y pausas, permitiendo controlar la capacidad operativa del sistema.

“Validación y normalización de datos críticos (Email Value Object)”

- ✓ Garantiza formato correcto de email

- ✓ Normaliza entradas del usuario (trim + lowercase)

- ✓ Evita datos inválidos desde el inicio del dominio

```
describe('Email', () => {  
  it('should create a valid email', () => {  
    const email = Email.create('test@example.com');  
    expect(email.value).toBe('test@example.com');  
  });  
  
  it('should normalize email to lowercase and trim it', () => {  
    const email = Email.create('  TEST@example.com  ');  
    expect(email.value).toBe('test@example.com');  
  });  
});
```

“Seguridad de identidad con Value Objects (UUIDs tipados)”

- ✓ Valida IDs en formato UUID

- ✓ Evita IDs inválidos en el sistema

- ✓ Encapsula reglas de integridad del dominio

- ✓ Mejora seguridad y consistencia de datos

```
describe('UserId, CustomerId, TenantId', () => {  
  const validUuid = '123e4567-e89b-12d3-a456-426614174000';  
  const invalidUuid = 'not-a-uuid';  
  
  it('should create valid IDs', () => {  
    const userId = UserId.create(validUuid);  
    const customerId = CustomerId.create(validUuid);  
    const tenantId = TenantId.create(validUuid);  
  
    expect(userId.value).toBe(validUuid);  
    expect(customerId.value).toBe(validUuid);  
    expect(tenantId.value).toBe(validUuid);  
  });  
});
```

```
PS C:\Users\pablo\Desktop\Proy_SW505_2026_1_Grupo_6\src\backend> npm run test:unit
```

```
> uni-booking-api@1.0.0 test:unit
```

```
> jest --config jest.config.cjs
```

```
PASS tests/unit/domain/entities/entities.spec.ts
```

```
PASS tests/unit/application/phase2-availability.spec.ts
```

```
PASS tests/unit/application/handlers.spec.ts
```

```
PASS tests/unit/domain/value-objects/value-objects.spec.ts
```

```
PASS tests/unit/application/phase2-handlers.spec.ts
```

```
PASS tests/unit/domain/entities/phase2-domain.spec.ts
```

```
Test Suites: 6 passed, 6 total
```

```
Tests: 81 passed, 81 total
```

```
Snapshots: 0 total
```

```
Time: 1.235 s
```

```
Ran all test suites.
```

```
PS C:\Users\pablo\Desktop\Proy_SW505_2026_1_Grupo_6\src\backend> npm run test:integration
```




Fase 4:

Notificaciones

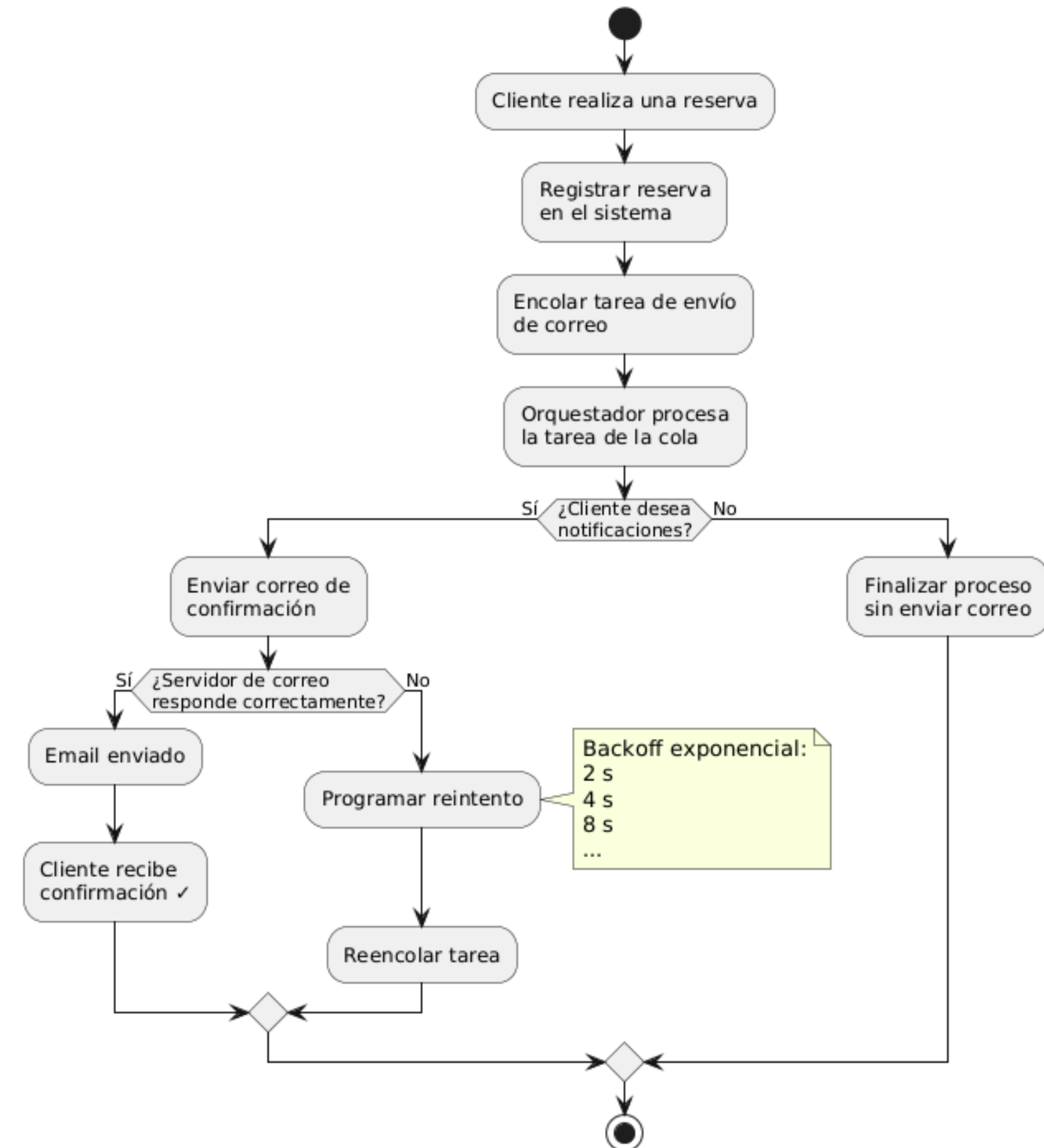
FLUJO GENERAL

Cuando un cliente hace una reserva — de un restaurante, una consulta médica, lo que sea — necesita recibir una confirmación por correo electrónico.

Yo implementé el módulo que se encarga de eso. El flujo es este: la reserva se registra, el sistema encola una tarea de envío, y de forma independiente procesa esa cola y manda el correo.

Si el servidor de correo falla — por ejemplo, hay un problema en la red — el sistema lo reintenta automáticamente, esperando un poco más cada vez. Voy a mostrarles el código de cada parte

Flujo de Confirmación de Reservas por Correo



1. NotificationEvent — la ficha de la notificación

Por cada correo que el sistema necesita enviar, se crea una 'ficha' llamada NotificationEvent. Esa ficha tiene la información de la reserva: a quién va el correo, el asunto, el contenido, y cuántas veces se ha intentado enviar. Aquí vemos dos funciones de esa ficha.

```
recordFailure(error: string): void {  
    if (this._retryCount >= this._maxRetries) {  
        throw new MaxRetriesExceededException(...);  
    }  
    this._retryCount++;  
    this._lastError = error;  
    this._status = NotificationEventStatus.failed();  
}  
  
getNextRetryDelay(): number {  
    return Math.pow(2, this._retryCount) * 1000;  
    // 1er intento fallido: espera 2000ms  
    // 2do intento fallido: espera 4000ms  
    // 3er intento fallido: espera 8000ms  
}
```

1. NotificationEvent — la ficha de la notificación

- **recordFailure** se llama cuando el envío falla. Primero verifica si ya se agotaron los intentos permitidos. Si sí, lanza un error y detiene los reintentos. Si no, incrementa el contador y guarda el mensaje de error, para que quede registro de qué salió mal
- **getNextRetryDelay** calcula cuánto tiempo esperar antes del próximo intento. Usa la fórmula 2 elevado al número de intentos. Así el sistema espera cada vez más tiempo, para no colapsar un servidor que ya está en problemas.

```
recordFailure(error: string): void {  
    if (this._retryCount >= this._maxRetries) {  
        throw new MaxRetriesExceededException(...);  
    }  
    this._retryCount++;  
    this._lastError = error;  
    this._status = NotificationEventStatus.failed();  
}
```

```
getNextRetryDelay(): number {  
    return Math.pow(2, this._retryCount) * 1000;  
    // 1er intento fallido: espera 2000ms  
    // 2do intento fallido: espera 4000ms  
    // 3er intento fallido: espera 8000ms  
}
```

2. SmtplibEmailService — el que envía el correo

Este componente es el que realmente manda el correo al cliente que hizo la reserva.

Recibe el email del cliente, el asunto — por ejemplo 'Confirmación de reserva #1234' — y el contenido HTML del correo.

Tiene tres detalles importantes.

```
async send(to: string, subject: string, html: string) {  
  try {  
    const info = await this.transporter.sendMail({  
      from:    process.env.SMTP_FROM,  
      to,  
      subject,  
      html,  
      text: html.replace(/<[^>]*>/g, ''),  
    });  
    return { messageId: info.messageId };  
  } catch (error) {  
    throw new EmailSendException(to, subject, error);  
  }  
}
```


2. SmtplibEmailService — el que envía el correo

- El remitente viene de una variable de entorno, no del código. Así el sistema puede usarse con cualquier dirección de correo según el negocio que lo use.
- Genera automáticamente una versión en texto plano para clientes de correo que no muestren HTML.
- Si el servidor de correo rechaza el envío, lanza una excepción con todos los datos del intento fallido, para que el sistema pueda registrarlo y reintentar."

```
async send(to: string, subject: string, html: string) {  
  try {  
    const info = await this.transporter.sendMail({  
      from:    process.env.SMTP_FROM,  
      to,  
      subject,  
      html,  
      text: html.replace(/<[^>]*>/g, ''),  
    });  
    return { messageId: info.messageId };  
  } catch (error) {  
    throw new EmailSendException(to, subject, error);  
  }  
}
```


3. El Orquestador — el que coordina todo

Este es el orquestador. Es el componente que coordina todo el flujo. Procesa las notificaciones pendientes una por una. Para cada reserva que necesita notificación hace tres cosas

```
for (const event of pendingEvents) {  
  
  // ¿El cliente de la reserva quiere recibir emails?  
  const prefs = await this.prefRepository.findByCustomerId(event.recipientId);  
  const canalActivo = prefs.some(p => p.channelId === 'email' && p.isEnabled);  
  if (!canalActivo) { continue; } // si deshabilitó, omitir  
  
  // Enviar el correo de confirmación  
  await this.emailService.send(  
    event.contactPoint, // email del cliente  
    event.subject,      // asunto del correo  
    event.renderedContent // HTML del correo  
  );  
  
  // Marcar la notificación como enviada en la base de datos  
  await this.markAsSentHandler.execute({ notificationEventId: event.id.value });  
}
```

3. El Orquestador — el que coordina todo

- Verifica las preferencias del cliente. Nuestro sistema permite que un cliente diga que no quiere recibir correos. Si ese es el caso, el sistema lo respeta y no manda nada.
- Si el cliente sí quiere el correo, llama al servicio SMTP para enviarlo. Acá vemos claramente qué se le pasa: el email de contacto del cliente, el asunto, y el contenido HTML del correo.
- Una vez enviado, actualiza la ficha de la notificación en la base de datos, marcándola como 'enviada' y registrando la hora exacta del envío

```
for (const event of pendingEvents) {

    // ¿El cliente de la reserva quiere recibir emails?
    const prefs = await this.prefRepository.findByCustomerId(event.recipientId);
    const canalActivo = prefs.some(p => p.channelId === 'email' && p.isEnabled);
    if (!canalActivo) { continue; } // si deshabilitó, omitir

    // Enviar el correo de confirmación
    await this.emailService.send(
        event.contactPoint, // email del cliente
        event.subject,      // asunto del correo
        event.renderedContent // HTML del correo
    );

    // Marcar la notificación como enviada en la base de datos
    await this.markAsSentHandler.execute({ notificationEventId: event.id.value });
}
```

3. CreateNotificationCommand Handler — crear la notificación

Este es el handler que da inicio al proceso. Cuando una reserva es confirmada en el sistema, este handler crea la ficha de notificación y la guarda en la base de datos. Por defecto son 3 intentos, como vemos en la línea `maxRetries ?? 3`. Al final devuelve el ID de la notificación creada, para que el sistema pueda rastrearla

```
async execute(command: CreateNotificationCommand): Promise<string> {  
  
  const event = NotificationEvent.create({  
    tenantId:      command.tenantId,    // el negocio que usa el sistema  
    recipientId:   command.recipientId, // el cliente que hizo la reserva  
    contactPoint:  command.contactPoint, // email del cliente  
    maxRetries:    command.maxRetries ?? 3, // máximo 3 intentos  
  });  
  
  await this.notificationEventRepository.save(event);  
  
  return event.id.value; // ID para rastrear la notificación  
}
```

4. Test unitario: verificar los reintentos

Para garantizar que el sistema funciona correctamente, escribimos pruebas automáticas.

Esta es una prueba unitaria — prueba un solo componente de forma aislada, sin base de datos ni servidor real.

El escenario que simula es este: el servidor de correo falla tres veces seguidas al intentar enviar la confirmación de una reserva.

```
it('debe calcular el tiempo de espera: 2^retryCount * 1000ms', () => {  
  const event = NotificationEvent.create(baseProps());  
  
  event.recordFailure('Error al conectar con el servidor de correo');  
  expect(event.getNextRetryDelay()).toBe(2000); // espera 2 segundos ✓  
  
  event.recordFailure('Servidor de correo sin respuesta');  
  expect(event.getNextRetryDelay()).toBe(4000); // espera 4 segundos ✓  
  
  event.recordFailure('Timeout de conexión');  
  expect(event.getNextRetryDelay()).toBe(8000); // espera 8 segundos ✓  
});
```

La prueba verifica que después del primer fallo el sistema espere 2 segundos antes de reintentar, después del segundo fallo espere 4 segundos, y después del tercero 8 segundos.

4. Test de integración: el flujo completo de una reserva

Esta es la prueba de integración del flujo completo.
Simula el escenario real: un cliente acaba de hacer una reserva y el sistema necesita enviarle el correo de confirmación.

La prueba tiene tres partes.

```
it('debe enviar la confirmación y marcar la notificación como enviada', async () => {

  // Un cliente acaba de hacer una reserva
  const event = makeEvent(); // notificación pendiente de envío
  mockEventRepo.findPendingEvents.mockResolvedValue([event]);
  mockEventRepo.findById.mockResolvedValue(event);

  // El orquestador procesa la cola
  await orchestrator.processQueue();

  // Verificar que se envió el correo correcto
  expect(mockEmailSvc.send).toHaveBeenCalledWith(
    'cliente@example.com', // destinatario: el cliente
    'Reserva Confirmada',  // asunto
    '<h1>Contenido</h1>',  // cuerpo del correo
  );
  // Verificar que la ficha quedó como "enviada"
  expect(event.status.value).toBe('sent');
  expect(event.sentAt).not.toBeNull(); // fecha y hora del envío registrada
});
```


Primero preparamos el escenario: creamos la notificación pendiente, como si acabara de generarse al confirmar la reserva.

Luego ejecutamos el orquestador completo, igual a como corre en producción.

Finalmente verificamos tres cosas: Que el correo se intentó enviar al email del cliente. Que el asunto y contenido son los correctos. Y que la ficha de la notificación quedó marcada como 'enviada', con la fecha y hora exactas registradas.

```
it('debe enviar la confirmación y marcar la notificación como enviada', async () => {

  // Un cliente acaba de hacer una reserva
  const event = makeEvent(); // notificación pendiente de envío
  mockEventRepo.findPendingEvents.mockResolvedValue([event]);
  mockEventRepo.findById.mockResolvedValue(event);

  // El orquestador procesa la cola
  await orchestrator.processQueue();

  // Verificar que se envió el correo correcto
  expect(mockEmailSvc.send).toHaveBeenCalledWith(
    'cliente@example.com', // destinatario: el cliente
    'Reserva Confirmada',  // asunto
    '<h1>Contenido</h1>', // cuerpo del correo
  );
  // Verificar que la ficha quedó como "enviada"
  expect(event.status.value).toBe('sent');
  expect(event.sentAt).not.toBeNull(); // fecha y hora del envío registrada
});
```

4. RESULTADOS

Esta es la captura real de cuando corrimos todas las pruebas unitarias.

Cada línea que ven es un archivo de pruebas que pasó al 100%: las fichas de notificaciones, las preferencias de los clientes, los handlers, el servicio de correo.

Resultado: 216 pruebas individuales, ninguna falla, y todo corre en menos de 2 segundos

```
PASS tests/unit/domain/entities/notification-event.spec.ts
PASS tests/unit/domain/entities/notification-preference.spec.ts
PASS tests/unit/domain/value-objects/notification-value-objects.spec.ts
PASS tests/unit/application/notification-handlers.spec.ts
PASS tests/unit/infrastructure/smtp-email.service.spec.ts
```

```
Test Suites: 15 passed, 15 total
```

```
Tests:      216 passed, 0 failed
```

```
Time:       1.836 s
```


4. RESULTADOS

Y esta es la captura de las pruebas de integración, donde probamos el flujo completo.

Ven los 20 escenarios que cubrimos: el caso normal de una reserva confirmada, el cliente que deshabilitó sus notificaciones, múltiples reservas procesadas en orden, el servidor de correo que falla, y el sistema que no manda duplicados.

20 escenarios, 20 aprobados, ningún fallo

```
PASS tests/integration/phase4-notifications-e2e.spec.ts
```

```
NotificationOrchestrationService – Integración
```

```
✓ enviar confirmación y marcar como enviada      (18ms)
✓ respetar preferencias: cliente que no quiere emails (1ms)
✓ omitir correo si el cliente deshabilitó el canal (1ms)
✓ procesar múltiples reservas en orden            (2ms)
✓ registrar fallo cuando el servidor de correo falla (12ms)
✓ no enviar duplicados si ya se está procesando    (3ms)
✓ calcular tiempo de espera: 2^reintentos × 1000ms (1ms)
```

```
Tests: 20 passed, 0 failed
```

Muchas Gracias!

